

Lectures on Monte Carlo Theory

Chapter 2: The Theory of Generators

Based on Leskeä & Vihola

Chapter 2 Outline

- 1 2.1 Introduction
- 2 2.2 Random Permutations
- 3 2.3 Pseudorandom Number Generators in Python
- 4 2.4 Simple Generators: Historical Perspective
- 5 2.5 Testing Good Properties of PRNGs
- 6 2.6 Bibliographical Notes

2.1 Introduction I

Randomness is a cornerstone of Monte Carlo methods. In practice, any algorithm that relies on randomness requires a source of *independent, identically distributed uniform random numbers*. These are used in simulating stochastic processes, estimating integrals by sampling, generating cryptographic keys, and performing statistical resampling. A digital computer, however, is a deterministic machine: it cannot produce genuinely random output. Instead, a computer runs a deterministic algorithm that produces a sequence whose *statistical properties* closely mimic those of a truly random sequence. Such an algorithm is called a **pseudorandom number generator (PRNG)**.

A motivating failure: the Middle-Square Method (von Neumann, 1940s). The idea is simple: start with a 4-digit integer (the “seed”), square it, extract the four middle digits, and repeat. For example, starting from 43:

$$43^2 = 1849, \quad 84^2 = 7056, \quad 05^2 = 0025, \quad 02^2 = 0004, \quad 00^2 = 0000 \rightarrow 0000 \rightarrow 0000 \rightarrow \dots$$

The sequence collapses to 0000 and stays there forever. This illustrates the core difficulty: any deterministic generator operating on a *finite* state space must eventually revisit a state and then cycle. Without careful design the cycle can be embarrassingly short.

2.1 Introduction II

Definition 2.1.1 — Pseudorandom Number Generator (PRNG)

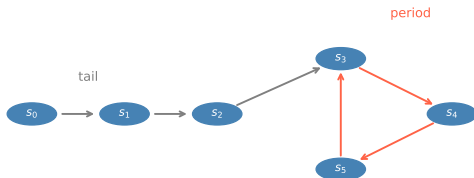
A PRNG is a 5-tuple (S, s_0, f, V, g) where

- S is a finite **state space**,
- $s_0 \in S$ is the **initial state** (the *seed*),
- $f : S \rightarrow S$ is the **transition function**: the state evolves as $s_{n+1} = f(s_n)$,
- V is a finite set of **output values**,
- $g : S \rightarrow V$ is the **output function**: the i -th output is $u_i = g(s_i)$, for $i = 0, 1, 2, \dots$

Because S is finite, the sequence $(s_n)_{n \geq 0}$ must eventually revisit a previous state and then repeat from that point forward. The length of the repeating cycle is called the **period** of the generator. Longer periods are desirable because the sequence takes more steps before repeating. The period cannot exceed $|S|$, the total number of states.

2.1 Introduction III

PRNG state sequence: tail + periodic cycle



2.1 Introduction IV

The figure above shows schematically how a PRNG visits states: after a “tail” leading into the cycle, the generator repeats with its period. A well-designed generator has a period close to $|S|$.

Key Insight

The quality of a PRNG depends critically on its period. A period much smaller than $|S|$ indicates a poorly designed generator. For a simulation requiring n samples, the period must be much larger than n ; otherwise the sequence simply repeats and introduces spurious correlations.

2.1 Introduction V

Definition 2.1.1.1 — Three Types of Random Sequences

Sequence of random numbers from $[0, 1)$. We say $U_1, U_2, \dots$ is a sequence of random numbers from $[0, 1)$ if each U_j is a random variable with the *uniform distribution* on $[0, 1)$:

$$\mathbb{P}(U_j \leq t) = \begin{cases} 0 & t < 0, \\ t & 0 \leq t < 1, \\ 1 & t \geq 1, \end{cases}$$

and the variables are *independent and identically distributed (i.i.d.)*:

$$\mathbb{P}(U_1 \leq t_1, \dots, U_n \leq t_n) = t_1 t_2 \cdots t_n.$$

We write $U \sim \mathcal{U}[0, 1)$ to denote this distribution.

Sequence of random bits. The B_j are i.i.d. taking values in $\{0, 1\}$ with $\mathbb{P}(B_j = 0) = \mathbb{P}(B_j = 1) = \frac{1}{2}$, so $\mathbb{P}(B_1 = b_1, \dots, B_n = b_n) = 1/2^n$ for any bit string.

Sequence of random integers from $\bar{M}$. The Y_j are i.i.d. uniformly distributed on $\{0, 1, \dots, M-1\}$. We write $Y \sim \mathcal{U}(\bar{M})$.

These three representations are interconvertible. Given $x_i \in [0, 1)$, set $x_i = \lfloor Mx_i \rfloor$ to obtain an integer

2.2 Random Permutations I

A **random permutation** of $\{1, \dots, n\}$ is a random variable σ taking values in the symmetric group $\mathcal{S}_n$ such that every arrangement has equal probability $1/n!$. Generating a uniformly random permutation is a fundamental subroutine in many Monte Carlo algorithms: it underlies random shuffling, randomised experiments (assigning treatments to units), bootstrapping, and permutation-based statistical tests.

The naive approach of picking elements one at a time uniformly at random from the full set $\{1, \dots, n\}$ until all n distinct elements appear is slow (its expected running time diverges logarithmically). The *Fisher-Yates shuffle* avoids this by building the permutation position by position, at each step selecting only from elements not yet placed.

2.2 Random Permutations II

Algorithm 1: Fisher-Yates Shuffle (Knuth Shuffle)

Require: Integer n ; pseudorandom numbers $\mathbf{u} = (u_1, \dots, u_n)$ with each $u_i \in [0, 1)$.

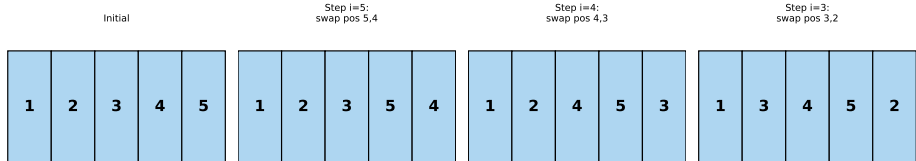
Ensure: A uniformly random permutation of $\{1, \dots, n\}$.

- 1 Initialise: $S[i] \leftarrow i$ for $i = 1, \dots, n$.
- 2 **for** $i = n$ **downto** 1 **do**
- 3 $k \leftarrow 1 + \lfloor i u_i \rfloor$ (picks a uniformly random index in $\{1, \dots, i\}$)
- 4 **Swap**($S[k], S[i]$)
- 5 **end for**
- 6 **return** S

The key idea: at step i we choose one of the i unplaced elements uniformly at random and move it into position i . By induction, after all n steps every ordering is equally likely with probability $1/n!$. The algorithm runs in $O(n)$ time (one swap per step).

2.2 Random Permutations III

Fisher-Yates Algorithm: Shuffling [1, 2, 3, 4, 5]



2.2 Random Permutations IV

Algorithm 2: Perm-Sort

An alternative method assigns a uniform random “key” to each index and then sorts by key:

- ① $S[i] \leftarrow i$ for $i = 1, \dots, n$.
- ② Generate $u_1, \dots, u_n$ i.i.d. $\mathcal{U}[0, 1)$.
- ③ Sort S so that $u_{S[1]} \leq u_{S[2]} \leq \dots \leq u_{S[n]}$.
- ④ **return** S .

Both algorithms produce a uniform random permutation given a truly uniform input sequence. The Fisher-Yates algorithm is preferred in practice because it runs in $O(n)$ time; Perm-Sort takes $O(n \log n)$ time because of the sort step, but its index-based structure can sometimes be easier to parallelise.

Correctness of Fisher-Yates: a brief argument. We prove by induction that after the loop has processed positions $n, n-1, \dots, i$, the elements in positions $i, i+1, \dots, n$ form a uniformly random arrangement of the corresponding elements. When $i = n$: we select position k uniformly from $\{1, \dots, n\}$ and swap $S[k]$ with $S[n]$, so the element in position n is uniform over $\{1, \dots, n\}$. Inductively, when we process position i , the remaining i elements in positions $1, \dots, i$ are equally likely

2.2 Random Permutations V

to be any i -element subset in any order, and selecting one of them uniformly to fill position i maintains the invariant. After the loop, every arrangement of $\{1, \dots, n\}$ has probability $1/n!$.

2.2 Random Permutations VI

2.2.2 Shuffling a Deck of 52 Cards. A standard deck of $n = 52$ cards can be identified with $\{1, \dots, 52\}$. A *card-shuffling scheme* is a Markov chain $(X_k)_{k \geq 0}$ on $\mathcal{S}_{52}$ whose stationary distribution is the uniform distribution (i.e., all $52! \approx 8.07 \times 10^{67}$ orderings equally likely). The key question is: how many steps are needed to reach approximate uniformity?

The distance from uniformity after k steps is measured by the **total variation distance**:

$$d_k = \frac{1}{2} \sum_{\sigma \in \mathcal{S}_n} \left| \mathbb{P}(X_k = \sigma) - \frac{1}{n!} \right|.$$

Here $d_k = 0$ would mean the deck is perfectly uniform, and $d_k = 1$ means the current arrangement is deterministic. As $k \rightarrow \infty$ the chain mixes and $d_k \rightarrow 0$.

Common shuffling methods:

Top-To-Random: at each step, remove the top card and insert it at a uniformly random position among the n available slots. This is the simplest scheme to analyse mathematically.

2.2 Random Permutations VII

Riffle Shuffle (Gilbert-Shannon-Reeds model): cut the deck into two halves, where the cut position follows a binomial distribution; then interleave by repeatedly dropping a card from whichever pile is larger, with probability proportional to pile size. This is the shuffle most people perform physically.

Time-Reversed Riffle Shuffle: the time-reversed operation interleaves two packs by independently inserting each card of one pack into the other; it is useful theoretically because its transitions are easier to compute.

Key Result (Bayer and Diaconis, 1992)

For a standard deck of $n = 52$ cards, approximately **7 riffle shuffles** are needed before the total variation distance d_k drops below $\frac{1}{2}$. With 6 shuffles the deck is still statistically far from random; with 8 it is essentially fully mixed. This surprising “cut-off phenomenon” was one of the landmark results in the theory of Markov chain mixing times.

2.3 Pseudorandom Number Generators in Python I

Modern scientific computing in Python uses the `numpy.random` module, which provides a family of high-quality PRNGs through a clean, object-oriented API. The recommended entry point is `numpy.random.default_rng(seed)`, which returns a Generator object backed by the **PCG64** algorithm. In this section we describe the available generators and their properties.

2.3 Pseudorandom Number Generators in Python II

2.3.1 The NumPy Generator Hierarchy

The NumPy API separates *bit generators* (which produce raw random bits) from *generators* (which transform those bits into useful distributions).

- `default_rng(seed)` creates a Generator using PCG64. This is the recommended choice for all new code.
- **PCG64** (Permuted Congruential Generator, 64-bit): period 2^{128} ; passes all standard test batteries; supports multiple independent streams via the `advance` method.
- **MT19937** (Mersenne Twister): period $2^{19937} - 1$; state space of 624 32-bit integers; historically dominant but fails some TestU01 BigCrush tests; used by Python's built-in `random` module and legacy NumPy (`np.random.seed`).
- **Philox** and **SFC64**: alternative high-quality generators that support multiple independent streams and parallel simulations.

Why PCG64? Its period of 2^{128} is so large that it exceeds the total number of random numbers any simulation could ever require. To put it in perspective: at 10^9 samples per second, exhausting a period

2.3 Pseudorandom Number Generators in Python III

of 2^{128} would take far longer than the age of the universe. PCG64 achieves this via a 128-bit linear congruential generator combined with a nonlinear output *permutation* that destroys the lattice structure inherent in simple LCGs (see Section 2.4.1).

MT19937: although its period $2^{19937} - 1$ is even larger, MT19937 has a very large state (624 32-bit integers), which makes it slow to initialise. It also fails some tests in TestU01 BigCrush and is not cryptographically secure. It should not be used for new projects.

2.3 Pseudorandom Number Generators in Python IV

Converting Between Output Types

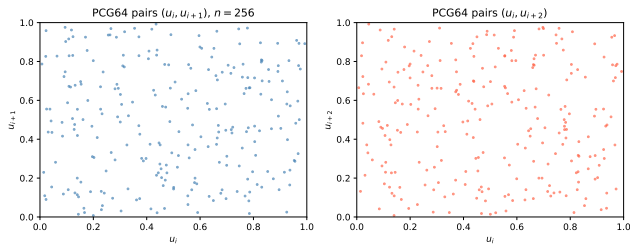
Given a uniform number $u_i \in [0, 1)$ from the generator, we convert as follows. To obtain an integer in $\{0, \dots, M - 1\}$, we use the floor function:

$$y_i = \lfloor Mu_i \rfloor.$$

For example, if $u_i = 0.731$ and $M = 6$, then $y_i = \lfloor 6 \times 0.731 \rfloor = \lfloor 4.386 \rfloor = 4$.

To obtain a random bit, we first convert u_i to an integer y_i with $M = 2^k$, then read the individual bits of y_i . The naive approach of setting $b_i = \mathbf{1}[u_i \geq 0.5]$ is dangerous: it discards all bits of u_i except the most significant one. A generator could pass this test while setting all remaining bits to zero, which would fail any proper bit-level test.

2.3 Pseudorandom Number Generators in Python V



The figure above shows 10 000 pairs (u_i, u_{i+1}) produced by PCG64. The points fill the unit square densely and uniformly, with no visible structure. This contrasts sharply with the lattice patterns visible in LCG pairs (Section 2.4.1).

2.3.1 Python Code: Using NumPy PRNGs

```
import numpy as np
from numpy.random import PCG64, MT19937, Generator

# --- Recommended: PCG64 via default_rng -----
rng = np.random.default_rng(seed=42)

u      = rng.random(10)           # 10 uniform numbers in [0, 1)
y      = rng.integers(0, 10, 10)  # 10 integers in {0, ..., 9}
perm   = rng.permutation(52)     # random permutation of {0,...,51}
print("Uniform:", u[:5])
print("Integers:", y[:5])
print("Permutation:", perm[:8], "...")

# --- Explicitly choose bit generator -----
rng_pcg = Generator(PCG64(seed=0))
rng_mt  = Generator(MT19937(seed=0))
print("PCG64: ", rng_pcg.random(5))
print("MT19937:", rng_mt.random(5))

# --- Multiple independent streams (PCG64 advance) -----
# Advance the second stream by 2^64 steps so that the two streams
# are statistically independent even when run in parallel.
bg1 = PCG64(seed=0)
bg2 = PCG64(seed=0)
bg2.advance(2**64)                # jump ahead 2^64 steps
rng1 = Generator(bg1)
rng2 = Generator(bg2)
```

2.3.2 Cryptographic PRNGs and RC4 I

For *cryptographic* applications, passing statistical tests is not sufficient. A cryptographic PRNG (CSPRNG) must be *computationally unpredictable*: even if an adversary observes a long prefix of the output stream, no polynomial-time algorithm should be able to predict the next bit with probability significantly better than $1/2$. Formally, a PRNG is **cryptographically secure** if no polynomial-time distinguisher can separate its output from truly random bits with non-negligible advantage.

Why statistical testing alone is not enough. A statistical test battery such as TestU01 only checks whether the output *looks* uniform and independent. It does not prevent an adversary from recovering the internal state from the output and thereby predicting all future values. CSPRNGs close this gap by requiring that state recovery be computationally infeasible under standard hardness assumptions (e.g., the difficulty of factoring large integers).

RC4 (Rivest Cipher 4) is a stream cipher designed by Ron Rivest in 1987. It became the workhorse of early web security (SSL/TLS, WEP wireless encryption, PDF encryption) because of its extreme simplicity and speed. RC4 operates on a permutation S of $\{0, \dots, n-1\}$ (typically $n = 256$) and generates one output byte per step. It consists of two phases: the Key-Scheduling Algorithm (KSA), which initialises S from a secret key, and the Pseudo-Random Generation Algorithm (PRGA), which streams output bytes.

2.3.2 Cryptographic PRNGs and RC4 II

Algorithm 3: RC4 Key-Scheduling Algorithm (KSA)

Require: integer n (typically 256); key $K[0], \dots, K[L - 1]$ (bytes).

Ensure: pseudorandom permutation $S[0], \dots, S[n - 1]$.

- ① Initialise: $S[i] \leftarrow i$ for $i = 0, \dots, n - 1$.
- ② Set $j \leftarrow 0$.
- ③ **for** $i = 0$ **to** $n - 1$ **do:** $j \leftarrow (j + S[i] + K[i \bmod L]) \bmod n$; **Swap**($S[i], S[j]$).
- ④ Reset $i \leftarrow 0, j \leftarrow 0$.

The key mixes the initial identity permutation. With $n = 256$, this produces a pseudorandom permutation of $\{0, \dots, 255\}$ that depends on the key.

2.3.2 Cryptographic PRNGs and RC4 III

Algorithm 4: RC4 Pseudo-Random Generation Algorithm (PRGA)

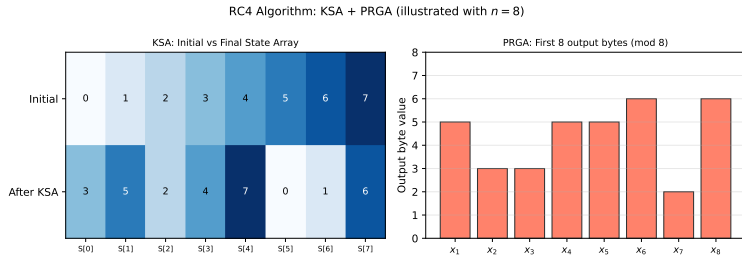
Require: permutation S from KSA.

Ensure: output byte stream $x_1, x_2, \dots$

- 1 Set $i \leftarrow 0, j \leftarrow 0, r \leftarrow 0$.
- 2 **while** generating output **do**:
 - $i \leftarrow (i + 1) \bmod n$
 - $j \leftarrow (j + S[i]) \bmod n$
 - **Swap**($S[i], S[j]$)
 - $x_r \leftarrow S[(S[i] + S[j]) \bmod n]$
 - $r \leftarrow r + 1$

Each output byte is derived from the permutation state after two index updates and one swap. The permutation evolves with every byte produced.

2.3.2 Cryptographic PRNGs and RC4 IV



The figure above illustrates the KSA and PRGA of RC4 side by side. In the KSA, the key byte $K[i \bmod L]$ contributes to the index j , mixing the key throughout the initial permutation. In the PRGA, both i and j advance on every step, causing every position in S to participate in the output.

Security weaknesses and deprecation. Despite its elegance, RC4 has serious statistical biases. The first few hundred output bytes are correlated with the key, and Fluhrer, Mantin, and Shamir (2001) showed that WEP's use of key-derived initialisation vectors allowed full key recovery from captured traffic. The IETF deprecated RC4 in RFC 7465 (2015). Modern protocols use AES in counter mode

2.3.2 Cryptographic PRNGs and RC4 V

(AES-CTR) or the ChaCha20 stream cipher instead, both of which have formal security proofs under standard hardness assumptions.

Key Insight

RC4 illustrates that a clever permutation-based design is not sufficient for cryptographic security. Modern CSPRNGs (AES-CTR, ChaCha20) are built on primitives whose security is formally reducible to the hardness of well-studied computational problems.

2.3.2 Python Code: RC4 Implementation

```
def rc4_ksa(n, key):
    """Key-Scheduling Algorithm: initialise permutation from key."""
    S = list(range(n))
    j, L = 0, len(key)
    for i in range(n):
        j = (j + S[i] + key[i % L]) % n
        S[i], S[j] = S[j], S[i]
    return S

def rc4_prng(S, num_bytes):
    """Pseudo-Random Generation Algorithm: produce keystream bytes."""
    n, i, j = len(S), 0, 0
    output = []
    while len(output) < num_bytes:
        i = (i + 1) % n
        j = (j + S[i]) % n
        S[i], S[j] = S[j], S[i]
        t = (S[i] + S[j]) % n
        output.append(S[t])
    return output

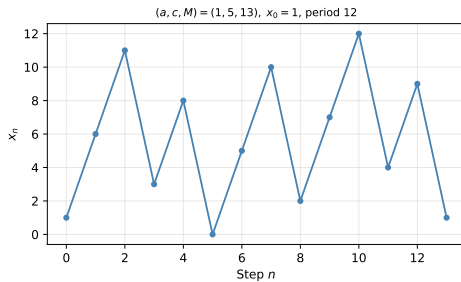
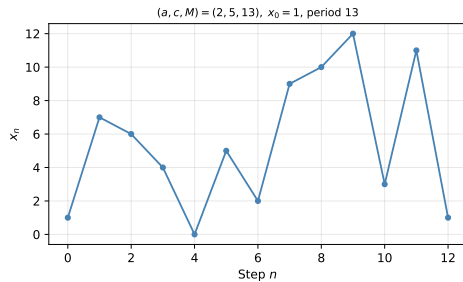
# Example: n=256 (standard), key = [3, 1, 4, 1, 5, 9]
key      = [3, 1, 4, 1, 5, 9]
S        = rc4_ksa(256, key)
stream   = rc4_prng(S, 20)
print("RC4 keystream (first 20 bytes):", stream)
```

2.4 Simple Generators: Historical Perspective I

Before the development of modern high-quality PRNGs, several simple generator families were widely used in commercial software and scientific computing. They are now primarily of historical and educational value. They are easy to implement and analyse mathematically, but they are typically 32-bit generators with limited periods, poor statistical quality by modern standards, and vulnerability to predictability attacks. Understanding them provides intuition for the challenges of PRNG design.

The fundamental challenge: lattice structure. Consider the LCG with recurrence $x_j = (137x_{j-1} + 187) \bmod 256$. This generator has period 256. If we plot the 256 pairs (u_i, u_{i+1}) in the unit square, the points do not fill the square uniformly. Instead, they lie on a small number of parallel lines — the **lattice structure** inherent in all LCGs. This lattice is immediately visible to the eye and represents a serious failure of two-dimensional uniformity.

2.4 Simple Generators: Historical Perspective II



2.4.1 Linear Congruential Generators (LCG) I

The **linear congruential generator (LCG)** is the simplest useful PRNG. It produces a sequence of integers by applying a linear recurrence modulo a large integer. Its appeal is its minimal computational cost: one multiplication, one addition, and one modulo operation per step.

The Linear Congruential Recurrence (Eq. 2.4.1)

$$x_{n+1} = (a x_n + c) \bmod M.$$

Here $M > 0$ is the **modulus** (the range of output integers), $0 \leq a < M$ is the **multiplier**, $0 \leq c < M$ is the **increment**, and $0 \leq x_0 < M$ is the **seed**. The output is $u_n = x_n/M \in [0, 1)$. When $c = 0$ the generator is called a *linear multiplicative congruential generator (LMCG)*.

Because the sequence (x_n) takes values in $\{0, \dots, M-1\}$, its period cannot exceed M . The goal is to choose parameters so the period is *exactly* M . The following theorem (from Knuth's *Art of Computer Programming*) gives necessary and sufficient conditions.

2.4.1 Linear Congruential Generators (LCG) II

Theorem 2.4.1 — Full-Period Conditions

Let $b = a - 1$. The LCG sequence has period M if and only if all three conditions hold simultaneously:

- 1 $\gcd(c, M) = 1$, i.e., c is coprime with M .
- 2 $b = a - 1$ is divisible by every prime p that divides M .
- 3 If $4 \mid M$, then $4 \mid b$, i.e., $a \equiv 1 \pmod{4}$.

Condition 1 says the increment c must share no common factors with the modulus; otherwise the output values would all be multiples of $\gcd(c, M)$ and the period would be reduced. Condition 2 ensures that the multiplier a is close enough to 1 modulo every prime divisor of M so that every residue class can be reached. Condition 3 is needed only when $4 \mid M$ to handle the factor of 2 that appears when M is divisible by a high power of 2.

2.4.1 Linear Congruential Generators (LCG) III

Example 2.4.2 (Classical 32-bit LCG)

$M = 2^{31} - 1 = 2,147,483,647$ (a Mersenne prime), $a = 7^5 = 16,807$, $c = 0$. Since M is prime, $c = 0$ is allowed (LMCG), and the period is $M - 1$. This was among the most widely used generators on 32-bit computers and was suggested by Lewis et al. [126]. Another popular 32-bit choice: $M = 2^{31} - 1$, $a = 40,014$. For 36-bit machines: $M = 2^{35} - 31$, $a = 5^5$.

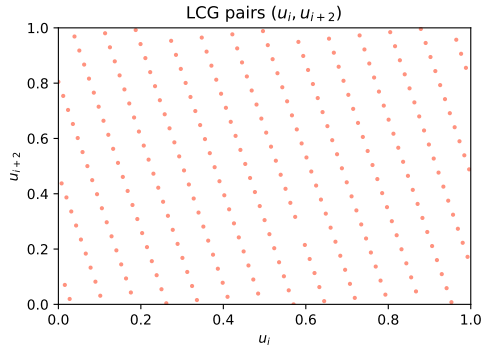
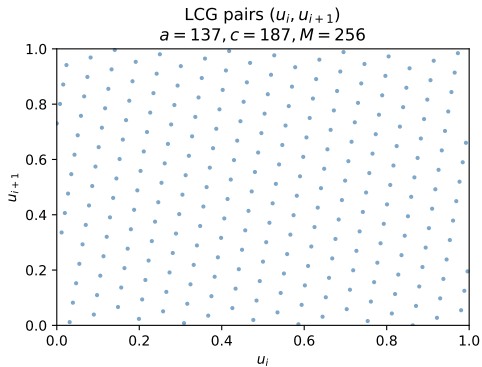
Example 2.4.3 (Resing's small LCG)

Take $(a, c, M) = (1, 5, 13)$, $x_0 = 1$. Here $b = 0$, $\gcd(5, 13) = 1$, and $M = 13$ is odd so condition 3 is vacuous. Theorem 2.4.1 is satisfied, and the sequence visits every element of $\{0, 1, \dots, 12\}$ exactly once:

1, 6, 11, 3, 8, 0, 5, 10, 2, 7, 12, 4, 9, 1, ... period 13.

Now take $(a, c, M) = (2, 5, 13)$, $x_0 = 1$. Here $b = 1$, and $13 \nmid 1$, so condition 2 fails. Indeed the sequence starting at $x_0 = 1$ has period 12 (not 13), and starting at $x_0 = 8$ produces the constant sequence 8, 8, 8, ... (period 1).

2.4.1 Linear Congruential Generators (LCG) IV



2.4.1 Linear Congruential Generators (LCG) V

The figure shows (u_i, u_{i+1}) pairs for a small LCG. The lattice structure is unmistakable: points lie on a small number of parallel lines rather than filling the square uniformly. This is **Marsaglia's theorem** (1968): the d -dimensional vectors $(u_i, u_{i+1}, \dots, u_{i+d-1})$ produced by any LCG always lie on at most $M^{1/d}$ parallel hyperplanes. For large M and small d the hyperplanes may be densely spaced and the structure is subtle, but it is always present.

2.4.1 Linear Congruential Generators (LCG) VI

Commercial LCG parameters historically in use.

System	Recurrence	Output formula
Java	$x_{i+1} = (25,214,903,917 x_i + 11) \bmod 2^{48}$	$u_i = (2^{27} \lfloor x_{2i}/2^{22} \rfloor + \lfloor x_{2i+1}/2^{21} \rfloor)$
rand48 (UNIX)	same x_i recurrence	$u_i = x_i/2^{48}$
Visual Basic	$x_i = (1,140,671,485 x_{i-1} + 12,820,163) \bmod 2^{24}$	$u_i = x_i/2^{24}$
Excel	$u_i = (0.9821 u_{i-1} + 0.211327) \bmod 1$	direct

All of these fail modern statistical test batteries (TestU01 BigCrush) and should *not* be used for serious Monte Carlo simulations.

2.4.1 Linear Congruential Generators (LCG) VII

Generalisations of the linear congruential method.

The **generalised linear congruential generator (GLCG)** uses a higher-order recurrence:

$$x_n = (a_1x_{n-1} + a_2x_{n-2} + \cdots + a_kx_{n-k} + c) \bmod M.$$

When M is prime and the coefficients a_j are chosen so the characteristic polynomial is a primitive polynomial over $\text{GF}(M)$, the period can reach M^k , far exceeding the M limit of a simple LCG.

Quadratic and cubic generators. The *Quadratic Congruential Generator I (QCG I)* is $x_{i+1} = x_i^2 \bmod M$ where M is a 512-bit prime. The *Quadratic Congruential Generator II (QCG II)* is $x_{i+1} = 2x_i^2 + 3x_i + 1 \bmod 2^{512}$. The *Cubic Congruential Generator (CCG)* uses $x_{i+1} = 2x_i^3 \bmod 2^{512}$. These provide longer periods and better theoretical properties but are slower to compute.

Fibonacci and Lagged Fibonacci generators. The Fibonacci generator is $x_n = x_{n-1} + x_{n-2} \bmod M$. The lagged Fibonacci variant due to Knuth uses

$$x_n = x_{n-k} + x_{n-l} \bmod M, \quad n \geq \max(k, l),$$

2.4.1 Linear Congruential Generators (LCG) VIII

with recommended parameters $k = 100$, $l = 37$, $M = 2^{30}$.

L'Ecuyer Mixed Generator. To achieve very long periods, L'Ecuyer combines two GLCG sequences. The combined output has a period approximately equal to the *product* of the two component periods, and the lattice structures from the two components partially cancel each other. The generator is defined by:

① $x_n = (A_1 x_{n-2} - A_2 x_{n-3}) \bmod M_1$

② $y_n = (B_1 y_{n-1} - B_2 y_{n-3}) \bmod M_2$

③ $z_n = (x_n - y_n) \bmod M_1$

④ Output: $u_n = z_n / (M_1 + 1)$ if $z_n > 0$; otherwise $u_n = M_1 / (M_1 + 1)$

2.4.1 Linear Congruential Generators (LCG) IX

Parameters: $M_1 = 4,294,967,087$, $M_2 = 4,294,944,443$, $A_1 = 1,403,580$, $A_2 = 810,728$, $B_1 = 527,612$, $B_2 = 1,370,589$. The period is approximately 2^{191} .

Key Insight

Combining two imperfect generators typically yields a better generator than either component, because the lattice structures of the two components are offset relative to each other. This “combination” principle underlies all composite PRNGs.

2.4.1 Python Code: LCG Implementation and Lattice

```
import numpy as np
import matplotlib.pyplot as plt

def lcg(a, c, M, x0, n):
    """Linear Congruential Generator: return n uniform outputs."""
    x, seq = x0, []
    for _ in range(n):
        x = (a * x + c) % M
        seq.append(x / M)
    return seq

# Example 2.4.3: (a,c,M) = (1,5,13), x0=1 => period 13
u = lcg(a=1, c=5, M=13, x0=1, n=14)
print("LCG (1,5,13), x0=1:", [f"{v:.3f}" for v in u])

# Example 2.4.3: (a,c,M) = (2,5,13), x0=1 => period 12 (not 13)
u2 = lcg(a=2, c=5, M=13, x0=1, n=14)
print("LCG (2,5,13), x0=1:", [f"{v:.3f}" for v in u2])

# Visualise lattice structure for a 256-state LCG
u256 = lcg(a=137, c=187, M=256, x0=1, n=256)
plt.figure(figsize=(5, 4))
plt.scatter(u256[:-1], u256[1:], s=4, alpha=0.7)
plt.title("LCG pairs: lattice structure")
plt.xlabel(r"$u_i$"); plt.ylabel(r"$u_{i+1}$")
plt.tight_layout(); plt.savefig("lcg_scatter.pdf")
```

2.4.2 Bit Generators I

Many PRNGs operate directly on *bits* rather than integers. A sequence of $\{0, 1\}$ -valued pseudorandom numbers is called a sequence of **pseudorandom bits**. Generators producing numbers in $[0, 1)$ can be converted to bits by representing each output in binary. Conversely, any bit sequence can produce $[0, 1)$ values by interpreting consecutive bits as a binary fraction. Cryptographic generators inherently operate at the bit level.

Register operations. On modern computers, bit sequences of length n are stored in n -bit registers (32-bit or 64-bit words). Two fundamental bitwise operations are used in every bit generator:

XOR and Bit Shifts

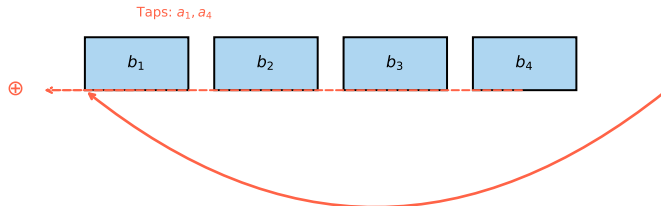
The **exclusive OR** ($\oplus$) returns 1 if and only if an odd number of its inputs equal 1. For single bits: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. For bit sequences: $\mathbf{p} \oplus \mathbf{q} = (p_1 \oplus q_1, \dots, p_n \oplus q_n)$.

The **left shift** $\ll a$ moves all bits a positions to the left, filling the a vacated positions on the right with zeros. The **right shift** $\gg b$ is the analogous shift to the right. For instance, $x = y \ll 2$ shifts y left by 2 bits.

2.4.2 Bit Generators II

Linear Feedback Shift Register (LFSR)

$$x_i = (a_1 x_{i-1} \oplus a_2 x_{i-2} \oplus \cdots \oplus a_k x_{i-k})$$



2.4.2 Bit Generators III

Exclusive OR Generator (XORG)

The XORG is one of the simplest bitwise generators. It leverages only the XOR operation to produce pseudorandom bits via the recurrence:

$$x_i = x_{i-1} \oplus x_{i-127}, \quad i \geq 128.$$

The generator is initialised with a 127-bit seed $x_1, \dots, x_{127}$. The XOR operation combines the most recent bit with a bit from 127 steps ago, producing the next bit. It is extremely fast (one XOR per output bit) and easy to implement in hardware, but its period and statistical quality are limited.

2.4.2 Bit Generators IV

Linear Feedback Shift Register (LFSR)

A more general linear bitwise generator uses a k -bit shift register with *feedback taps* to produce the recurrence:

$$x_i = a_1x_{i-1} \oplus a_2x_{i-2} \oplus \cdots \oplus a_kx_{i-k},$$

where $a_j \in \{0, 1\}$ are the feedback coefficients. The LFSR has period at most $2^k - 1$ (it cannot output the all-zeros state). When the feedback polynomial $a_1 + a_2x + \cdots + a_kx^{k-1}$ is a *primitive polynomial* over $\text{GF}(2)$, the LFSR achieves this maximal period of $2^k - 1$. LFSRs are extremely fast and are widely used in hardware (FPGAs, communication chips) because they require only a shift register and a few XOR gates. However, they are *not* cryptographically secure: given only $2k$ consecutive output bits, one can solve a linear system to recover all tap coefficients and then predict every future output bit.

2.4.2 Bit Generators V

Blum-Blum-Shub (BBS) Generator

The BBS generator is the canonical example of a *provably cryptographically secure* bit generator:

$$x_{i+1} = x_i^2 \bmod M, \quad M = p q,$$

where p and q are large primes both congruent to 3 modulo 4. The output bit at each step is the *least significant bit* of x_i . Security rests on the **quadratic residuosity assumption**: without knowing the factorisation of M , no polynomial-time algorithm can distinguish quadratic residues modulo M from non-residues with non-negligible probability. BBS is provably secure, but modular squaring is slow compared to XOR-based generators, making it impractical for high-throughput Monte Carlo use.

2.4.2 Bit Generators VI

Shift-XOR (SXR) Generator

The SXR generator combines shifts and XOR for a fast and reasonably good generator:

- 1 Choose seed $x_0 \in \{1, \dots, 2^n - 1\}$ (typically $n = 32$ or $n = 64$).
- 2 Compute: $x_{i+1} = (x_i \oplus (x_i \ll a)) \oplus (x_i \gg b)$, where a and b are shift parameters.
- 3 Output x_{i+1} .

For $n = 32$ with parameters $a = 13$, $b = 17$: starting from seed $x_0 = 0xABCDE123$:

$$x_1 = (x_0 \oplus (x_0 \ll 13)) \oplus (x_0 \gg 17), \quad x_2 = (x_1 \oplus (x_1 \ll 13)) \oplus (x_1 \gg 17), \quad \dots$$

Properties: simple bitwise implementation; period depends on shift parameters; good statistical quality for simple uses but may fail advanced statistical tests.

2.4.2 Bit Generators VII

Key Insight

The three bitwise generators form a security/speed spectrum. LFSR is the fastest but is trivially broken by a linear algebra attack. BBS is provably secure but far too slow for Monte Carlo use. SXR is a practical middle ground with good speed and statistical quality for non-cryptographic applications.

2.5 Testing Good Properties of PRNGs I

The quality of a PRNG directly determines the reliability of any simulation, integral estimate, or statistical analysis built on it. Statistical testing is how we gain confidence that a generator's output is indistinguishable from truly random output for practical purposes.

There are two complementary approaches to assessing PRNG quality:

Theoretical tests examine the internal structure of the generator without running it. The *lattice test* analyses the geometry of the point set $\{(u_i, u_{i+1}, \dots, u_{i+d-1}) : i = 0, 1, \dots\}$ in $[0, 1)^d$, and the *spectral test* measures the minimum distance between parallel hyperplanes that contain all these points (a good generator should have small hyperplane spacing in all dimensions).

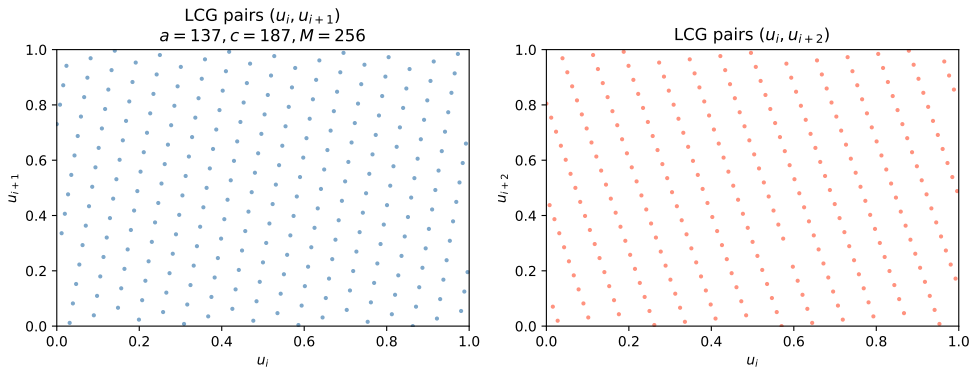
Statistical tests treat the PRNG as a black box and apply hypothesis tests to the output sequence. They answer the question: could this sequence plausibly have been produced by a truly random source? If the answer is consistently “no”, the generator is considered defective.

Major test batteries:

- **Diehard** (Marsaglia): 18 tests; widely used historically.
- **TestU01** (L'Ecuyer and Simard): SmallCrush (10 tests), Crush (96 tests), BigCrush (106 tests).
The most demanding battery in current use.

2.5 Testing Good Properties of PRNGs II

- **NIST SP 800-22:** 15 tests designed specifically for cryptographic applications.



The figure above shows the motivating example: pairs (u_i, u_{i+1}) from a simple 256-state LCG lie on a few parallel lines, immediately failing a visual test for 2D uniformity. A good generator's pairs would fill the square densely.

2.5.1 Preliminary Remarks I

Before describing specific tests we establish notation and address two practical matters: how to convert between output formats, and how to organise a sequence into multidimensional vectors for testing.

Notation. Throughout this chapter we use capital letters for random variables and lowercase letters for their realisations:

- $U \sim \mathcal{U}[0, 1)$: uniformly distributed on $[0, 1)$. Realisation: u .
- $B \sim \mathcal{U}(\{0, 1\})$: a uniform random bit. Realisation: b .
- $Y \sim \mathcal{U}(\bar{M})$: uniformly distributed on $\{0, \dots, M - 1\}$. Realisation: y .

2.5.1 Preliminary Remarks II

2.5.1.1 Converting Numbers

Given a sequence $u_1, u_2, \dots$ in $[0, 1)$, convert to integers via

$$y_i = \lfloor Mu_i \rfloor. \quad (2.5.1)$$

Conversely, $u_i = y_i/M$ recovers a $[0, 1)$ value.

To convert to bits, first convert u_i to $y_i \in \{0, \dots, 2^k - 1\}$ using $M = 2^k$, then extract the binary bits of y_i .

Why the naive threshold $b_i = 1[u_i \geq 0.5]$ is dangerous. If $u_i = (0.b_1b_2 \dots b_{24})_2$ is a 24-bit float from Visual Basic's LCG, the threshold rule keeps only bit b_1 and discards $b_2, \dots, b_{24}$. A generator that sets $b_2 = \dots = b_{24} = 0$ (making all odd outputs zero) would produce a binary sequence that passes the threshold test perfectly but would immediately fail any proper bit-level test.

2.5.1 Preliminary Remarks III

2.5.1.2 Grouping into Multidimensional Vectors

Given $n = mr$ numbers $u_1, \dots, u_n$, group them into r vectors of length m :

$$\mathbf{u}_1 = (u_1, \dots, u_m), \quad \mathbf{u}_2 = (u_{m+1}, \dots, u_{2m}), \quad \dots, \quad \mathbf{u}_r = (u_{(r-1)m+1}, \dots, u_{rm}).$$

Each $\mathbf{u}_i \in [0, 1)^m$. After converting via $y_j = \lfloor Mu_j \rfloor$, we obtain r integer vectors $\mathbf{y}_i \in \{0, \dots, M-1\}^m$. Similarly for bits: group $n = mr$ bits into r vectors $\mathbf{b}_i \in \{0, 1\}^m$. This multidimensional grouping is the framework used in virtually all test batteries.

2.5.1 Preliminary Remarks IV

2.5.1.3 The p -Value Concept

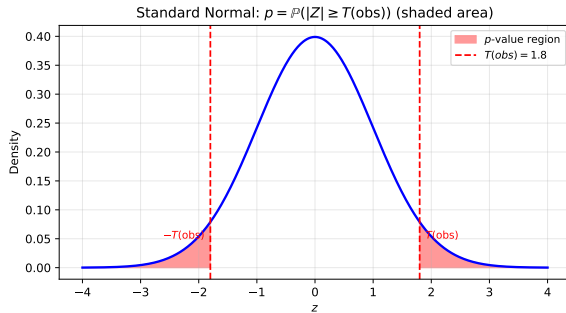
For a test statistic T with c.d.f. F under the null hypothesis $\mathcal{H}_0$ (“the sequence is truly random”), the p -value is

$$p = \mathbb{P}(T \geq T(\text{obs})) = 1 - F(T(\text{obs})).$$

Under $\mathcal{H}_0$ with a continuous statistic T , we have $p \sim \mathcal{U}[0, 1)$. Therefore:

- A small p (e.g., $p < 0.01$) is evidence *against* $\mathcal{H}_0$, meaning the sequence is unlikely to be random.
- A suspiciously *large* p (e.g., $p > 0.99$) also indicates non-randomness: the sequence is “too uniform” to be natural. This is exactly what happens with quasi-random (low-discrepancy) sequences.

2.5.1 Preliminary Remarks V

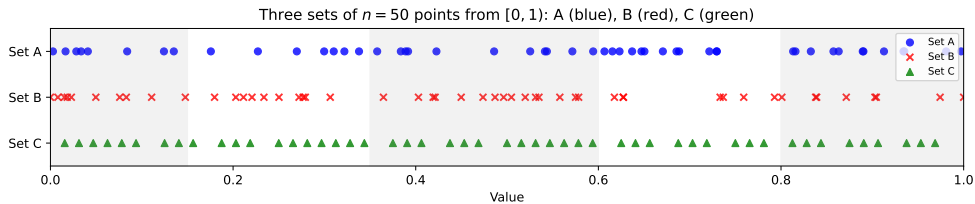


2.5.2 Kolmogorov-Smirnov and Chi-Square Tests I

We illustrate the two fundamental goodness-of-fit tests using three sets of $n = 50$ points from $[0, 1)$, defined as follows:

- **Set A** (blue): generated by PCG64 via NumPy's `default_rng`. This is the pseudorandom baseline and should pass all tests with typical p -values.
- **Set B** (red): a nonlinear, biased transform of Set A, given by $u_i^B = (e^{u_i^A - 1} - m)/(M - m)$ where m, M are the min/max of $\{e^{u_j^A - 1} - \varepsilon\}$ and $\varepsilon = 10^{-7}$. This set is skewed (concentrated near 0) and should be rejected.
- **Set C** (green): a quasi-random (Halton/van der Corput) sequence. It is “too uniformly distributed” to be random and should be detected by second-level testing (Section 2.5.6).

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests II



2.5.2 Kolmogorov-Smirnov and Chi-Square Tests III

2.5.2.1 Kolmogorov-Smirnov (KS) Test

The KS test compares the **empirical c.d.f.**

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(U_i \leq t)$$

with the theoretical c.d.f. $F(t) = t$ for $\mathcal{U}[0, 1)$. The **KS statistic** is the maximum absolute deviation:

$$D_n = \sup_{0 \leq t \leq 1} |\hat{F}_n(t) - t|.$$

By the Glivenko-Cantelli theorem, $D_n \rightarrow 0$ almost surely when the data are i.i.d. uniform. Moreover, the normalised statistic $\sqrt{n} D_n$ converges in distribution to the **Kolmogorov distribution** with c.d.f.

$$K(t) = \lim_{n \rightarrow \infty} \mathbb{P}(\sqrt{n} D_n \leq t) = 1 - 2J(\exp(-2t^2)), \quad t > 0,$$

where $J(t) = \sum_{j=1}^{\infty} (-1)^{j-1} \exp(-2j^2 t^2)$. Critical values at common significance levels: $\lambda_{0.1} = 1.224$, $\lambda_{0.05} = 1.358$, $\lambda_{0.01} = 1.628$.

2.5.2 Python Code: KS and Chi-Square Tests

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(seed=0)
n = 50
# Set A: PCG64 pseudorandom
A = rng.random(n)
# Set B: biased nonlinear transform of A
eps = 1e-7
raw = np.exp(A - 1) - 1 - eps
B = (raw - raw.min()) / (raw.max() - raw.min())
# Set C: quasi-random equispaced (approximates Halton sequence)
C = np.arange(1, n + 1) / (n + 1)

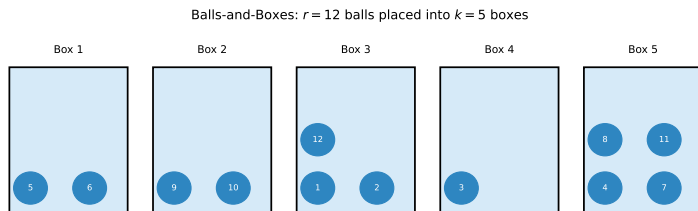
# --- KS test against U[0,1) ---
for name, seq in [("A", A), ("B", B), ("C", C)]:
    stat, p = stats.kstest(seq, 'uniform')
    print(f"KS Set {name}: D={stat:.4f}, p={p:.4f}")

# --- Chi-square test with 5 non-uniform bins from the book ---
bins = [0, 0.15, 0.35, 0.60, 0.80, 1.0]
probs = [0.15, 0.20, 0.25, 0.20, 0.20]
for name, seq in [("A", A), ("B", B), ("C", C)]:
    obs, _ = np.histogram(seq, bins=bins)
    exp = [n * p for p in probs]
    chi2, p_val = stats.chisquare(obs, exp)
    print(f"Chi2 Set {name}: X2={chi2:.3f}, p={p_val:.4f}")
```


2.5.3 Tests Based on Balls-and-Boxes Schemes I

Many classical PRNG tests can be cast as **balls-and-boxes problems**. In the general scheme, we place r balls into k boxes independently and uniformly at random (probability $1/k$ per box). Under the null hypothesis $\mathcal{H}_0$, the occupancy counts follow a multinomial distribution $M(r, 1/k, \dots, 1/k)$.

To convert a PRNG output sequence (u_j) into a balls-and-boxes scheme, form r vectors $\mathbf{u}_1, \dots, \mathbf{u}_r$ each of length m , then partition $[0, 1)^m$ into $k = L^m$ sub-hypercubes. Each $\mathbf{u}_i$ maps to exactly one sub-hypercube (“box”); these are the “balls landing in boxes”. The choice of m , L , and k defines the specific test.



2.5.3 Tests Based on Balls-and-Boxes Schemes II

The Birthday Problem. Consider r students whose birthdays are uniformly distributed among $k = 365$ days. The probability that all birthdays are distinct is

$$\frac{(365)_r}{365^r} = \prod_{j=1}^{r-1} \left(1 - \frac{j}{365}\right),$$

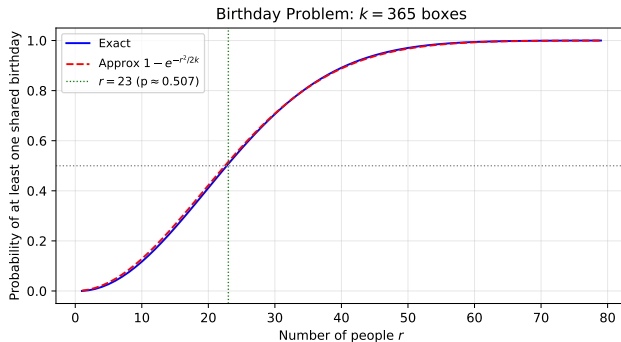
where $(365)_r = 365 \cdot 364 \cdots (365 - r + 1)$ is the falling factorial. The probability of at least one shared birthday (a “multiple-birth”) is $p = 1 - (365)_r/365^r$. With $r = 23$ students, $p \approx 0.507$; with $r = 70$, $p \approx 0.999$.

For general k boxes and r balls:

$$p_{k,r} = 1 - \prod_{j=1}^{r-1} \left(1 - \frac{j}{k}\right) \approx 1 - \exp\left(-\frac{r^2}{2k}\right), \quad (2.5.12)$$

where the approximation follows from $\log(1 - x) \approx -x$ for small x .

2.5.3 Tests Based on Balls-and-Boxes Schemes III



2.5.3 Tests Based on Balls-and-Boxes Schemes IV

Collision Test

A *collision* occurs when a ball lands in a box that already contains at least one other ball. Let $C_{k,r}$ denote the number of collisions after placing r balls in k boxes. The exact probability:

$$\mathbb{P}(C_{k,r} = c) = \frac{k(k-1) \cdots (k-r+c+1) S_2(r, r-c)}{k^r}, \quad (2.5.13)$$

where $S_2(r, s)$ is the Stirling number of the second kind (the number of ways to partition a set of r elements into s non-empty unordered subsets).

To understand this formula: c collisions means the r balls are partitioned into $r - c$ non-empty subsets. There are $S_2(r, r - c)$ ways to form such partitions, and $k(k-1) \cdots (k - (r - c) + 1)$ ordered ways to assign them to distinct boxes.

The **Poisson approximation**: when $r, k \rightarrow \infty$ with $r^2/(2k) \rightarrow \lambda$, we have

$$\mathbb{P}(C_{k,r} = c) \rightarrow \frac{\lambda^c}{c!} e^{-\lambda}. \quad (2.5.14)$$

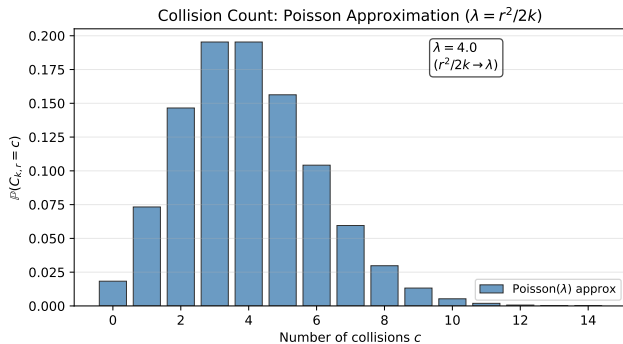
2.5.3 Tests Based on Balls-and-Boxes Schemes V

Expected Number of Collisions (Eq. 2.5.17)

$$\mathbb{E}C_{k,r} = r - k \left(1 - \left(1 - \frac{1}{k} \right)^r \right) \approx \frac{r^2}{2k}.$$

For $r = 2^{14}$ and $k = 2^{20}$: $\mathbb{E}C_{k,r} \approx 2^{28}/2^{21} = 128$.

2.5.3 Tests Based on Balls-and-Boxes Schemes VI



2.5.3 Tests Based on Balls-and-Boxes Schemes VII

Birthday Spacing Test (Marsaglia). This test is among the most stringent for LCGs. Given Rn bits, split them into R subsequences each of length $n = mr$ (with $m = 25$ and $r = 2^9$). For each subsequence, treat the r 25-tuples (interpreted as integers) as “birthdays” in a year of $k = 2^{25}$ days. Sort the birthdays and compute the spacings between consecutive sorted values; let K = number of equal spacings among consecutive sorted spacings. Under $\mathcal{H}_0$, K approximately follows a Poisson distribution with mean $\lambda = r^3/(4k)$.

Apply a chi-square test to the R values of K across the R subsequences. This test fails the MATLAB `rand('state')` LCG with p -value 4.12×10^{-100} .

Poker Test. Group $n = 5r$ integers $y_1, \dots, y_n \in \{0, \dots, M-1\}$ into r 5-tuples and classify each by the number of *distinct values* $s \in \{1, 2, 3, 4, 5\}$ (analogous to poker hands). The probability that a 5-tuple from $\mathcal{U}(\bar{M})$ has exactly s distinct values is

$$p_s = \frac{M(M-1) \cdots (M-s+1) S_2(5, s)}{M^5},$$

where $S_2(5, 1) = 1$, $S_2(5, 2) = 15$, $S_2(5, 3) = 25$, $S_2(5, 4) = 10$, $S_2(5, 5) = 1$. Apply a chi-square test with 4 degrees of freedom.

2.5.3 Tests Based on Balls-and-Boxes Schemes VIII

Key Insight

The balls-and-boxes framework unifies many seemingly different tests. The birthday spacing test is especially sensitive to LCGs because their lattice structure causes spacings to be non-uniform. Every test in this family is, at its core, checking whether the PRNG output fills a multi-dimensional hypercube uniformly.

2.5.4 Tests Based on Random Walks I

Random walk tests convert a bit sequence into a 1D random walk and then test properties of that walk whose distributions are known exactly under $\mathcal{H}_0$. Because the theory of simple random walks is very rich (CLT, arcsine law, excursion theory), these tests achieve precise p -value computation.

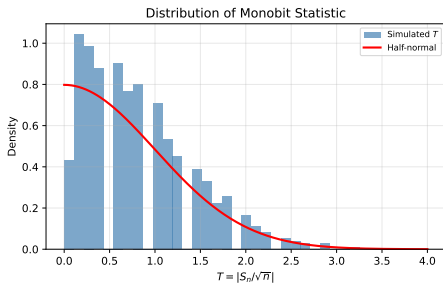
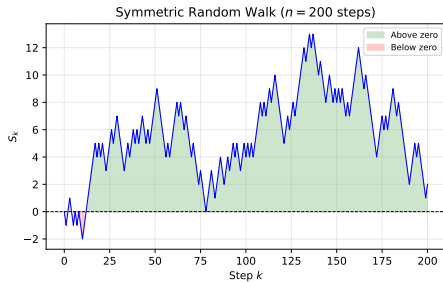
From Bits to Random Walk (Eq. 2.5.20)

Given bits $b_1, b_2, \dots$, define the centred increments and partial sums:

$$X_i = 2B_i - 1 \in \{-1, +1\}, \quad S_0 = 0, \quad S_k = \sum_{i=1}^k X_i.$$

Here $X_i = +1$ when $b_i = 1$ (step right) and $X_i = -1$ when $b_i = 0$ (step left). Under $\mathcal{H}_0$: $\mathbb{E}X_i = 0$, $\text{Var}X_i = 1$, and (S_k) is the *symmetric random walk* on $\mathbb{Z}$.

2.5.4 Tests Based on Random Walks II



2.5.4 Tests Based on Random Walks III

Frequency (Monobit) Test

By the Central Limit Theorem, $S_n/\sqrt{n} \xrightarrow{\mathcal{D}} \mathcal{N}(0, 1)$. The test statistic is $T = |S_n|/\sqrt{n}$, which follows a half-normal distribution under $\mathcal{H}_0$. The p -value is

$$p = \operatorname{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right),$$

where $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt$. The monobit test is the simplest test: it checks whether the number of 0s and 1s in the bit sequence is approximately equal. A biased bit stream will produce a large $|S_n|$, giving a small p -value. The approximation is valid for $n \geq 30$; in practice $n \geq 100$ is preferred.

2.5.4 Tests Based on Random Walks IV

Frequency Test in Blocks

Split $n = mr$ bits into r blocks of length m . For block i , let $S_m^{(i)}$ be the walk position at the end of the block. Define $Z_i = S_m^{(i)} / \sqrt{m}$. For large m , the Z_i are approximately i.i.d. $\mathcal{N}(0, 1)$, so

$$T(\text{obs}) = \frac{1}{m} \sum_{i=1}^r (S_m^{(i)})^2 \approx \chi_r^2.$$

This tests whether the balanced-bits property holds uniformly across all blocks, not just globally. A generator with a long-range trend in bit frequencies would pass the monobit test but fail the block frequency test.

2.5.4 Tests Based on Random Walks V

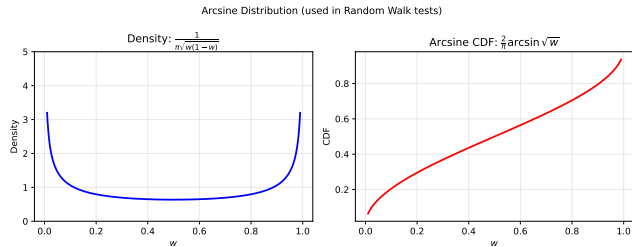
Arcsine Law Based Test

A remarkable property of the symmetric random walk is described by the **arcsine law**: the fraction of time the walk spends above zero is *not* concentrated near $1/2$ but instead follows an arcsine distribution. Formally, let L_{2r}^+ denote the number of steps during which the walk is strictly positive. Then:

$$\lim_{r \rightarrow \infty} \mathbb{P}\left(\frac{L_{2r}^+}{2r} \leq w\right) = \frac{2}{\pi} \arcsin \sqrt{w}, \quad w \in [0, 1].$$

The density $\frac{1}{\pi\sqrt{w(1-w)}}$ is U-shaped: the walk is most likely to spend almost all its time above zero or almost all its time below zero, with probability 0 of spending exactly half its time above. This counter-intuitive result (Feller, 1968) forms the basis of a powerful test: the p -value is $p \approx 1 - \frac{2}{\pi} \arcsin \sqrt{T(\text{obs})}$.

2.5.4 Tests Based on Random Walks VI



2.5.4 Tests Based on Random Walks VII

Random Excursion Test (NIST)

An *excursion* is a maximal segment of the walk between two consecutive returns to zero. Let J_n denote the number of zeros (returns to 0) in $S_1, \dots, S_n$. Under $\mathcal{H}_0$:

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\frac{J_n}{\sqrt{n}} \leq t\right) = \sqrt{\frac{2}{\pi}} \int_0^t e^{-x^2/2} dx.$$

NIST recommends rejecting $\mathcal{H}_0$ if $J_n(\text{obs}) \leq 0.008\sqrt{n}$ (too few returns to zero).

For a value $x \in \{1, 2, \dots, 7\}$, let $L(x)$ be the number of visits to state x during one excursion. The distribution of $L(x)$ under $\mathcal{H}_0$ is:

$$\mathbb{P}(L(x) = k) = \begin{cases} 1 - \frac{1}{2|x|}, & k = 0, \\ \frac{1}{4x^2} \left(1 - \frac{1}{2|x|}\right)^{k-1}, & k \geq 1. \end{cases} \quad (2.5.22)$$

We count observed visits $E_k(x)$ for $k = 0, 1, \dots, 4$ (merging $k \geq 5$) across all excursions, then apply a chi-square test with 5 degrees of freedom for each value of x .

2.5.5 Permutation Tests I

Permutation tests convert blocks of pseudorandom numbers into permutations and test statistical properties of those permutations that are known exactly under $\mathcal{H}_0$. They exploit the rich combinatorial theory of permutations (derangements, fixed points, cycle structure) to construct precise tests.

Derangements (Lemma 2.5.10)

A **derangement** of $\{1, \dots, m\}$ is a permutation σ with *no fixed points*: $\sigma(i) \neq i$ for all i . The number of derangements of $\{1, \dots, m\}$ is given by inclusion-exclusion:

$$d_m = m! \sum_{k=0}^m \frac{(-1)^k}{k!}.$$

As $m \rightarrow \infty$, $d_m/m! \rightarrow e^{-1} \approx 0.3679$. To derive this: let A_i be the event that $\sigma(i) = i$. Then by inclusion-exclusion, the number of permutations with *at least one* fixed point is $\sum_{k=1}^m (-1)^{k+1} \binom{m}{k} (m-k)!$, and $d_m = m!$ minus this.

2.5.5 Permutation Tests II

Worked example. For $m = 4$: $d_4 = 24(1 - 1 + 1/2 - 1/6 + 1/24) = 24 \cdot 3/8 = 9$. The 9 derangements of $\{1, 2, 3, 4\}$ are: $\{2134\}$ is *not* a derangement ($\sigma(3) = 3$, $\sigma(4) = 4$); ... one must verify all 24 permutations. The probability of a derangement is $9/24 \approx 0.375 \approx e^{-1}$.

2.5.5 Permutation Tests III

Derangement Permutation Test

Require: $u = (u_1, \dots, u_n)$, $n = mr$.

- 1 Set $\xi \leftarrow 0$.
- 2 **for** $i = 1$ to r : generate permutation σ_i from the i -th m -tuple via Fisher-Yates; if σ_i is a derangement, increment ξ .
- 3 Under $\mathcal{H}_0$: $\xi/r \approx e^{-1}$, so the normalised test statistic is

$$T(\text{obs}) = \frac{|\xi - re^{-1}|}{\sqrt{re^{-1}(1 - e^{-1})}}, \quad p = \text{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right).$$

2.5.5 Permutation Tests IV

Fixed-Point Permutation Test

For each m -tuple, generate permutation σ via Fisher-Yates and count the fixed points $\xi = \#\{j : \sigma(j) = j\}$.

Under $\mathcal{H}_0$: $\xi \xrightarrow{\mathcal{D}} \text{Poisson}(1)$ as $m \rightarrow \infty$. Group counts into 11 boxes $A_j = \{j\}$ for $j = 0, \dots, 9$, $A_{10} = \{10, 11, \dots\}$, with expected probabilities $p_j = e^{-1}/j!$ for $j \leq 9$ and $p_{10} = 1 - \sum_{j=0}^9 p_j$. Apply a chi-square test with 10 degrees of freedom.

2.5.5 Python Code: Permutation Tests

```
import numpy as np
from scipy import stats

def fisher_yates(m, u):
    """Generate permutation of {0,...,m-1} from m uniform inputs."""
    S = list(range(m))
    for i in range(m, 0, -1):
        k = int(i * u[i - 1])      # uniform in {0,...,i-1}
        S[k], S[i - 1] = S[i - 1], S[k]
    return S

def is_derangement(sigma):
    return all(sigma[i] != i for i in range(len(sigma)))

rng = np.random.default_rng(42)
m, r = 10, 2000                # 10-tuples, 2000 repetitions
n = m * r
u = rng.random(n)

# ---- Derangement test ----
xi = sum(is_derangement(fisher_yates(m, u[i * m:(i + 1) * m]))
         for i in range(r))
e_inv = np.exp(-1)
T_obs = abs(xi - r * e_inv) / np.sqrt(r * e_inv * (1 - e_inv))
p_val = stats.erfc(T_obs / np.sqrt(2))
print(f"Derangements: {xi}/{r}  T={T_obs:.3f}  p={p_val:.4f}")
```

2.5.6 p -Values and Second-Level Testing I

A single statistical test applied to a single sequence is often not decisive. A genuinely good PRNG might occasionally produce a small p -value by chance (type-I error), and a defective generator might pass a particular test for particular parameter values. **Second-level testing** addresses this by applying the test to *many* independent sequences and checking that the resulting p -values are themselves uniformly distributed.

p -Values are Uniform Under $\mathcal{H}_0$

Suppose T has a continuous, strictly increasing c.d.f. F under $\mathcal{H}_0$. The p -value is $p = 1 - F(T)$. Then:

$$\mathbb{P}(p \leq t) = \mathbb{P}(1 - F(T) \leq t) = \mathbb{P}(T \geq F^{-1}(1 - t)) = 1 - F(F^{-1}(1 - t)) = t.$$

So $p \sim \mathcal{U}[0, 1)$ under $\mathcal{H}_0$. Any deviation from uniformity in the distribution of p -values signals non-randomness: values clustering near 0 indicate a biased or correlated generator, while values clustering near 1 indicate the sequence is “too uniform” (quasi-random).

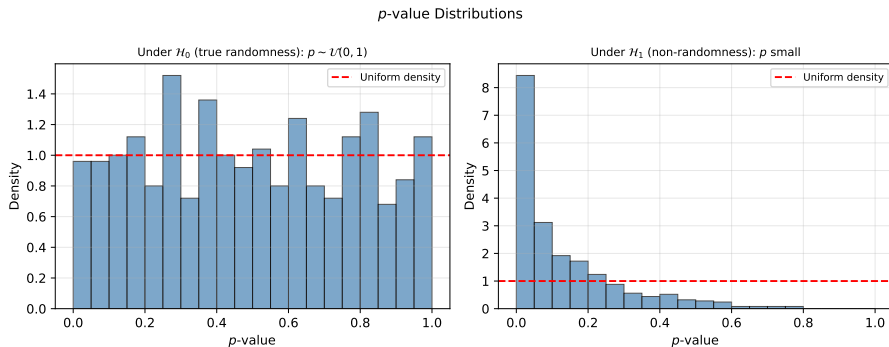
2.5.6 p -Values and Second-Level Testing II

Second-Level Testing Procedure:

- 1 Split a sequence of length Rn into R non-overlapping subsequences, each of length n .
- 2 For each subsequence i : compute test statistic $T^{(i)}$ and $p^{(i)} = 1 - F(T^{(i)})$.
- 3 Test whether $p^{(1)}, \dots, p^{(R)}$ are $\mathcal{U}[0, 1)$ by partitioning $[0, 1)$ into s equal bins $[(j-1)/s, j/s)$ for $j = 1, \dots, s$, counting $O_j = |\{i : p^{(i)} \in \text{bin } j\}|$ with expected $E_j = R/s$, and computing

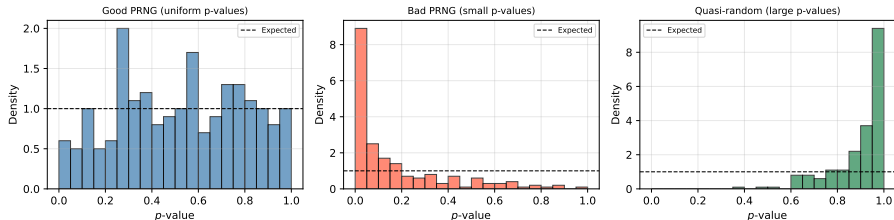
$$X_{\text{final}}^2 = \sum_{j=1}^s \frac{(O_j - R/s)^2}{R/s} \approx \chi_{s-1}^2.$$

2.5.6 p -Values and Second-Level Testing III



2.5.6 p -Values and Second-Level Testing IV

Second-Level Testing: Histograms of $R = 200$ p -values



2.5.6 p -Values and Second-Level Testing V

Practical recommendations (NIST SP 800-22). Use $1000 \leq R \leq 10000$ subsequences, each of length $2^{20} \leq n \leq 2^{32}$. Use significance level $\alpha \in [0.001, 0.01]$. The fraction of sequences that pass (i.e., have $p^{(i)} > \alpha$) should lie within three standard deviations of $1 - \alpha$:

$$\hat{B} = \frac{1}{R} \sum_{i=1}^R \mathbf{1}[p^{(i)} > \alpha] \in \left[1 - \alpha \pm 3\sqrt{\frac{\alpha(1 - \alpha)}{R}} \right].$$

Example 2.5.12: Second-Level Test on Sets A, B, C

Apply the KS test to $R = 200$ length- n subsequences for each of the three sets.

- **Set A** (PCG64): p -values are approximately uniform across $[0, 1)$ — passes second-level test.
- **Set B** (biased): p -values cluster near 0 because each subsequence exhibits the same leftward skew — *fails*.
- **Set C** (quasi-random): p -values cluster near 1 because each subsequence is “too uniform” — also *detected as non-random*, even though first-level KS did not reject it.

2.5.6 p -Values and Second-Level Testing VI

Key Insight

Second-level testing is strictly more powerful than first-level testing. It is the *only* reliable method to detect quasi-random sequences that fool first-level tests by being “too uniform”. NIST SP 800-22 mandates it as the primary validation method for cryptographic PRNGs.

2.5.6 Python Code: Second-Level Testing

```
import numpy as np
from scipy import stats

def second_level_test(sequences, test_fn, s=10):
    """
    Run test_fn on each sequence to get a p-value, then test whether
    those p-values are U[0,1) using a chi-square uniformity test.
    sequences : list of 1D arrays
    test_fn    : callable returning a p-value for one sequence
    s          : number of equal bins for the chi-square check
    """
    p_vals = [test_fn(seq) for seq in sequences]
    bins = np.linspace(0, 1, s + 1)
    obs, _ = np.histogram(p_vals, bins=bins)
    exp = [len(p_vals) / s] * s
    chi2, p_final = stats.chisquare(obs, exp)
    return np.array(p_vals), chi2, p_final

# --- Set A: PCG64 pseudorandom sequences -----
rng = np.random.default_rng(0)
R, n = 200, 50
seqs_A = [rng.random(n) for _ in range(R)]

# --- Set C: quasi-random shifted equispaced sequences -----
seqs_C = [(np.arange(n) / n + i / R) % 1.0 for i in range(R)]

def ks_pval(seq):
```

2.6 Bibliographical Notes I

This chapter draws on half a century of PRNG research. The following references are essential for deeper reading.

Foundational references:

- **Knuth** [107] *The Art of Computer Programming, Vol. 2*: the definitive mathematical treatment of LCGs, the full-period theorem (Theorem 2.4.1), the spectral test, Stirling numbers, the serial test, and the poker test. Essential reading for anyone designing or analysing PRNGs.
- **L'Ecuyer** [118, 122]: designed the combined generators of Section 2.4.1 and the TestU01 test suite. His TestU01 paper remains the benchmark reference for PRNG testing; BigCrush comprises 106 tests and is the most stringent battery in current use.
- **Marsaglia** [141, 145]: invented the Diehard test battery and the birthday spacing test; his 1968 paper proved the lattice structure of all LCGs.
- **NIST SP 800-22** [187]: the US government statistical test suite for cryptographic PRNGs; 15 standardised tests including the monobit test, block frequency test, and excursion test.

2.6 Bibliographical Notes II

Modern generators:

- **PCG64** (O'Neill [162]): Permuted Congruential Generators; the current NumPy default via `default_rng`. Uses a 128-bit LCG with a nonlinear output permutation that eliminates lattice structure while maintaining a period of 2^{128} .
- **MT19937** (Matsumoto and Nishimura [146]): Mersenne Twister; period $2^{19937} - 1$; passes Diehard but fails some TestU01 BigCrush tests. State size of 624 32-bit integers makes initialisation slow. Not suitable for cryptographic use.
- **AES-CTR, ChaCha20**: cryptographically secure stream ciphers replacing RC4 in modern protocols (RFC 7539, RFC 8439). Not covered here but important in practice for all cryptographic applications.

Shuffling and card mixing:

- **Bayer and Diaconis** [14]: proved the “7 shuffle theorem” for the riffle shuffle on 52 cards using Fourier analysis on $\mathcal{S}_{52}$.
- **Aldous and Diaconis** [2]: Markov chain coupling methods for mixing-time analysis; provides the theoretical foundation for Section 2.2.2.

2.6 Bibliographical Notes III

Theoretical results on tests:

- **Weiss** [210]: CLT for the empty-boxes count (Proposition 2.5.9).
- **Holst** [83] and **L'Ecuyer et al.** [122]: known results on the serial test chi-square statistic.
- **Tsang et al.** [205]: analysis of the random excursion test.
- **L'Ecuyer and Simard** [169, 170]: design of reliable tests using Berry-Esseen bounds to control normal-approximation errors.

2.7 Selected Exercises I

The following exercises reinforce the theoretical foundations of this chapter.

2.T.1 (Markov and Chebyshev inequalities). For a non-negative random variable X and any $a > 0$:

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}X}{a} \quad (\text{Markov}).$$

For a real-valued X with finite variance and any $b > 0$:

$$\mathbb{P}(|X - \mathbb{E}X| \geq b) \leq \frac{\text{Var}X}{b^2} \quad (\text{Chebyshev}).$$

Both inequalities follow from each other (Chebyshev applied to $(X - \mathbb{E}X)^2$).

2.T.4. Let $U_1, U_2, \dots$ be i.i.d. $\mathcal{U}[0, 1)$ and $M \geq 2$ an integer. Show that $Y_i = \lfloor MU_i \rfloor$ is a random sequence from $\bar{M} = \{0, \dots, M-1\}$, i.e., the Y_i are i.i.d. and $Y_i \sim \mathcal{U}(\bar{M})$. (Hint: compute $\mathbb{P}(Y_i = k)$ directly from $\mathbb{P}(Y_i = k) = \mathbb{P}(k/M \leq U_i < (k+1)/M) = 1/M$.)

2.7 Selected Exercises II

2.T.8. Verify that Fisher-Yates (Algorithm 1) produces a uniformly random permutation of $\{1, \dots, n\}$. (Hint: by induction on the swap steps, show that after step i , the element in position i is uniformly distributed over the i elements that have not yet been placed in higher positions.)

2.T.17 (Serial test mean). Let O_i ($i = 1, \dots, k$) be the number of random vectors $\mathbf{u}_j$ falling into box i in the serial test. Show that under $\mathcal{H}_0$: $\mathbb{E}X_k^2 = k - 1$.

2.7 Selected Exercises III

2.T.19 (Riffle Shuffle and Time Reversal). Let $p_{RS}(\sigma, \sigma')$ denote the transition probability of a single Riffle Shuffle step, and $p_{TR-RS}(\sigma, \sigma')$ the time-reversed Riffle Shuffle. Show that for any $\sigma, \sigma' \in \mathcal{S}_n$: $p_{RS}(\sigma, \sigma') = p_{TR-RS}(\sigma', \sigma)$.

Lab Exercises:

2.L.1. Implement the LCG with $M = 2^{11} - 1$, $a = 5$, $c = 1$. Generate 10^4 numbers and plot (i, u_i) . Find the period computationally and verify Theorem 2.4.1.

2.L.2. Compare Fisher-Yates (Algorithm 1) to a modified version where line 3 is replaced by $k = \text{random}(1, \dots, n)$ (drawing from the full range, not $\{1, \dots, i\}$). Test with the fixed-point permutation test: the modified version should fail because it does not produce a uniform permutation.

2.L.3. Generate $n = 10^5$ numbers each with MT19937 and the LMCG ($M = 2^{31} - 1$, $a = 16807$). Compute the Pearson autocorrelation between $(y_1, \dots, y_{n-1})$ and $(y_2, \dots, y_n)$. Which is better?

2.L.5. Quasi-random test: let $u_i = (i \cdot \phi) \bmod 1$ where $\phi = (\sqrt{5} - 1)/2$ is the golden ratio (a Kronecker low-discrepancy sequence). For $n = 10^6$ numbers, apply the KS, chi-square, frequency pairs, and second-level KS tests. Observe that first-level tests pass while second-level testing detects the quasi-randomness.

Chapter 2: Summary

Generators:

- PRNG is a 5-tuple (S, s_0, f, V, g) ; period $\leq |S|$.
- Three equivalent random sequence types: $\mathcal{U}[0, 1)$, bits, integers.
- Fisher-Yates shuffle: uniform permutation in $O(n)$.
- Python: `default_rng/PCG64` (period 2^{128}) and `MT19937` (period $2^{19937} - 1$).
- RC4: KSA initialises permutation; PRGA streams bytes; now deprecated in favour of AES-CTR/ChaCha20.
- LCG full-period theorem (Theorem 2.4.1).
- Bit generators: LFSR, XORG, SXR, BBS.
- Combined generators (L'Ecuyer): period $\approx 2^{191}$.

Statistical Testing:

- KS test: $D_n = \sup |\hat{F}_n - t|$, p -value via Kolmogorov distribution.
- Chi-square: $X^2 = \sum (O_i - E_i)^2 / E_i$, require $E_i \geq 5$.
- Serial (frequency of pairs): detects 2D correlations.
- Balls-and-boxes: birthday, collision, spacing, poker tests.
- Random walk tests: monobit, blocks, arcsine law, excursion.
- Permutation tests: derangement, fixed-point count.
- Second-level testing: p -values should be $\mathcal{U}[0, 1)$ across $R \geq 1000$ sequences.